

COURSE CODE/TITLE: CPE 010-CPE21S1 - Data Structures and Algorithms	SECTION: CPE21S1
DATE PERFORMED: Aug 18, 2026	DATE SUBMITTED: Aug 27, 2026
NAME: Mendoza, Roy Gabriel I.	INSTRUCTOR: Engr. Jimlord Quejado

ACTIVITY NO. 6.1	Searching Techniques
-------------------------	-----------------------------

1. PROCEDURE OUTPUT

What procedures did you perform?

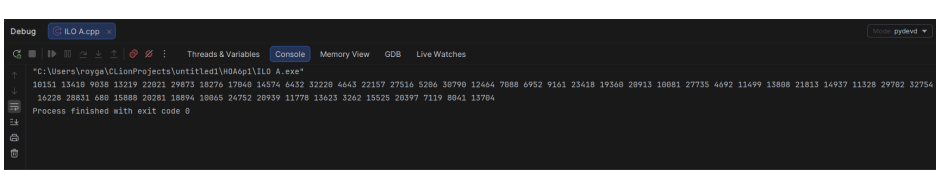
ILO A: Create C++ code for searching data using different techniques	
Screenshot	 The screenshot shows a C++ program running in a debugger. The console output displays a list of 50 random integers generated by the program. The integers are: 18151, 13410, 9038, 13219, 22021, 29873, 18276, 17048, 14574, 6432, 32228, 4663, 22157, 27316, 5286, 38790, 12464, 7088, 6952, 9161, 23418, 19368, 28913, 10881, 27735, 4692, 11499, 13808, 21813, 14937, 11328, 29782, 32754, 16228, 28831, 688, 15888, 20281, 18894, 18865, 24752, 20939, 11778, 13623, 3262, 15525, 20397, 7119, 8841, 13784. The process finished with exit code 0.
Observation	The output of the code that I copied/make from the online class that sir made generates an (X) number of random elements using the <code>[const int max_size = 50;]</code> for the basis on how many elements are to be generated.

Table 6-1. Data Generated and Observations

Code

Main

```
35 //linked list: R O Y G A B
36 Node<char>* name1 = new_node<char>( newData: 'R');
37 Node<char>* name2 = new_node<char>( newData: 'O');
38 Node<char>* name3 = new_node<char>( newData: 'Y');
39 Node<char>* name4 = new_node<char>( newData: 'G');
40 Node<char>* name5 = new_node<char>( newData: 'A');
41 Node<char>* name6 = new_node<char>( newData: 'B');
42 Node<char>* name7 = new_node<char>( newData: 'Z');
43
44 name1->next = name2;
45 name2->next = name3;
46 name3->next = name4;
47 name4->next = name5;
48 name5->next = name6;
49 name6->next = name7;
50 name7->next = nullptr;
51
52 while(true) {
53     std::cout<<"\n == Performing Linear Search== \n"<<"Enter a character you want to find: ";
54     std::cin>>charfind;
55
56     //loop for finding character inside until break
57     if (charfind == '!') {
58         std::cout<<"\nProgram Termination";
59         break;
60     }
61     linearsearchLL(name1, charfind);
62 }
63 return 0;
64
65 }
```

Header

```
25 //linked list approach
26 //Nodes -> starting node
27 //temp that we are searching
28 template <typename T>
29 void linearsearchLL(Node<T>* node, T dataFind) {
30     //repeat the loop until the current node is not null
31     while (node != nullptr) {
32         //if item = current node's data, then display searching is successful
33         if (dataFind==node->data) {
34             std::cout<<"The search is success\n";
35             return;
36         };
37         //if not, move to the next node
38         node = node->next;
39     }
40     //the searching reached nullptr
41     std::cout<<"The search has failed\n";
42 }
```

Output	<pre> "C:\Users\royga\CLionProjects\untitled1\H04Ap1\Aug 18 2026.exe" 31663 38338 13186 3169 18766 5527 23165 13753 28838 15023 2586 31107 8132 26418 23403 29828 2185 12084 22832 29583 15389 12582 25824 21598 23635 7638 30469 21535 31759 6890 16195 16553 23598 7714 26227 16334 27717 3833 31119 8428 6181 17615 10082 27839 29917 13682 31188 10185 18072 26861 What value you want to search?: 0 The search is failed == Performing Linear Search== Enter a character you want to find: R The search is success == Performing Linear Search== Enter a character you want to find: T The search has failed == Performing Linear Search== Enter a character you want to find: I Program Termination Process finished with exit code 0 </pre>
Observation	From my observation, it uses the traversal using the pointers instead of the array in 6.2a, so it can move through the memory.

Table 6-2b. Linear Search for Linked List

Code	<pre> 44 template <typename T> 45 Node<T>* getMiddle(Node<T>* start, Node<T>* last) { 46 if (start == nullptr) return nullptr; 47 48 Node<T>* slow = start; 49 Node<T>* fast = start; 50 51 while (fast != last && fast->next != last) { 52 slow = slow->next; 53 fast = fast->next->next; 54 } 55 return slow; 56 } 57 /* 58 *1. Start node is set to head of the list and the last node is set to NULL. 59 *2. Middle element is calculated using the two pointers approach discussed above. 60 *3. If the middle element is same as the key to be searched, we return it. 61 *4. Else if middle element is greater than the key to be searched, we have to search is the 62 * right side of the singly linked list. So, we set start pointer to the next of middle 63 * element. 64 *5. Else if middle element is less than the key to be searched, we have to search is the left 65 * side of the singly linked list. So, we set last pointer to the middle element. 66 *6. If the key is found or the entire linked list gets traversed, we break the loop. 67 */ </pre>
------	---

```

68     template <typename T>
69     Node<T>* binarySearchLL(Node<T>* start, T findData) {
70         Node<T>* last = nullptr; // exclusive end
71
72         while (start != last) {
73             Node<T>* mid = getMiddle(start, last);
74             if (mid == nullptr) break;
75
76             if (mid->data == findData) {
77                 std::cout << "The search is success\n";
78                 return mid;
79             } else if (mid->data < findData) {
80                 start = mid->next; // search right half
81             } else {
82                 last = mid; // search left half (exclusive)
83             }
84         }
85
86         std::cout << "The search has failed\n";
87         return nullptr;
88     }
89
90     template <typename T>
91     int binarySearch(T arr[], int n, T key) {
92         int low = 0;
93         int up = n - 1;
94
95         while (low <= up) {
96             int mid = (low + up) / 2;
97             if (arr[mid] == key) {
98                 std::cout << "Search element is found at index " << mid << "\n";
99                 return mid;
100             } else if (key < arr[mid]) {
101                 up = mid - 1;
102             } else {
103                 low = mid + 1;
104             }
105         }
106
107         std::cout << "Search element is not found\n";
108         return -1;
109     }
110 #endif //UNTITLED1_SEARCHINGALGO_H
111

```

Output	<pre>Program Termination9632 What value you want to search?: 9 Search element is not found Returned index: -1 26735 What value you want to search?: 6 Search element is not found Returned index: -1 18484 What value you want to search?: 21345 Search element is not found Returned index: -1 25494 What value you want to search?: 1681 Search element is not found Returned index: -1 1226 What value you want to search?:</pre>
Observation	For some reason, it would not find anything, even if I copy pasted the output of the dataset, maybe this is because it is not sorted yet.

Table 6-3a. Binary Search for Arrays

Code

```
81 //-----
82 char choice = 'y';
83 int count = 1;
84 int newData;
85 Node<int> *temp = nullptr, *head = nullptr, *node = nullptr;
86
87 while (choice == 'y') {
88     std::cout << "\nEnter data (ordered): ";
89     std::cin >> newData;
90
91     if (count == 1) {
92         head = new_node<int>(newData);
93         head->next = nullptr;
94         std::cout << "Successfully added " << head->data << " to the list.\n";
95         count++;
96     } else if (count == 2) {
97         node = new_node<int>(newData);
98         node->next = nullptr;
99         head->next = node;
100         std::cout << "Successfully added " << node->data << " to the list.\n";
101         count++;
102     } else {
103         temp = head;
104         while (temp->next != nullptr) temp = temp->next;
105         node = new_node<int>(newData);
106         node->next = nullptr;
107         temp->next = node;
108         std::cout << "Successfully added " << node->data << " to the list.\n";
109         count++;
110     }
111
112     std::cout << "Continue? (y/n): ";
113     std::cin >> choice;
114     if (choice == 'n') break;
115 }
116
117 // display the integer list
118 Node<int> *currNode = head;
119 std::cout << "\nList contents: ";
120 int nodeCount = 0;
121 while (currNode != nullptr) {
122     std::cout << currNode->data << " ";
123     currNode = currNode->next;
124     nodeCount++;
125 }
126 std::cout << "\nTotal nodes: " << nodeCount << "\n\n";
127
128 // perform binary search on the linked list
129 int key;
130 std::cout << "Enter value to binary-search in the list: ";
131 std::cin >> key;
132
```

	<pre> 132 133 Node<int>* found = binarySearchLL<int>(head, key); 134 if (found != nullptr) { 135 std::cout << "Returned node value: " << found->data << "\n"; 136 } else { 137 std::cout << "Returned node: nullptr (not found)\n"; 138 } 139 140 // cleanup allocated nodes for the integer list 141 currNode = head; 142 while (currNode != nullptr) { 143 Node<int>* next = currNode->next; 144 delete currNode; 145 currNode = next; 146 } 147 148 // cleanup allocated nodes for the char list 149 Node<char>* curc = name1; 150 while (curc != nullptr) { 151 Node<char>* nextc = curc->next; 152 delete curc; 153 curc = nextc; 154 } 155 return 0; 156 }</pre>
Output	<pre> "C:\Users\royga\CLionProjects\untitled1\H0A6p1\Aug 18 2026.exe" 13182 32528 17256 29351 23493 14538 8651 19214 13605 19755 What value you want to search?: 5 The search has failed == Performing Linear Search== Enter a character you want to find: ! Program Termination Now sorting the array and performing binary search. Sorted dataset: 8651 13182 13605 14538 17256 19214 19755 23493 29351 32528 What value you want to search (binary search)?: 8651 Search element is found at index 0 Returned index: 0 </pre> <pre> Enter data (ordered): 10 Successfully added 10 to the list. Continue? (y/n): y Enter data (ordered): 20 Successfully added 20 to the list. Continue? (y/n): y Enter data (ordered): 30 Successfully added 30 to the list. Continue? (y/n): y Enter data (ordered): 40 Successfully added 40 to the list. Continue? (y/n): n List contents: 10 20 30 40 Total nodes: 4 Enter value to binary-search in the list: 30 The search is success Returned node value: 30 </pre>
Observation	On this part of the activity, I put a sorting [insertionSort] which only works on integers so far. This is a must because it would result to an undefined and crashes, like when I enter a letter, it just loops infinitely. Since binary search requires the data to be sorted first before getting properly executed, this is a must for it to work.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PROBLEM 1:

CODE

```
1 //  
2 // Created by royga on 8/23/2026.  
3 //  
4  
5 #ifndef UNTITLED1_NODES_H  
6 #define UNTITLED1_NODES_H  
7  
8 template <typename T>  
9 class Node {  
10 public:  
11     T data;  
12     Node<T> *next;  
13 };  
14  
15 // CREATING NEW NODE  
16 template <typename T>  
17 Node<T>* new_node(T newData) {  
18     Node<T>* newNode = new Node<T>(); // single pointer  
19     newNode->data = newData;  
20     newNode->next = nullptr;  
21     return newNode;  
22 }  
23  
24  
25 #endif //UNTITLED1_NODES_H
```

searchingalgo.h x nodes.h

```
1 > //...
2
3
4
5 #ifndef UNTITLED1_SEARCHINGALGO_H
6 #define UNTITLED1_SEARCHINGALGO_H
7 > #include ...
8
9
10 // Linear search arr
11 template <typename T>
12 void linearSearchArr(T data[], int arrSize, T dataFind) {
13     for (int i = 0; i < arrSize; ++i) {
14         if (dataFind == data[i]) {
15             std::cout << "The search is success\n";
16             return;
17         }
18     }
19     std::cout << "The search has failed\n";
20     return;
21 }
22
23 // Linear search linkedli
24 template <typename T>
25 void linearsearchLL(Node<T>* node, T dataFind) {
26     Node<T>* cur = node;
27     while (cur != nullptr) {
28         if (cur->data == dataFind) {
29             std::cout << "The search is success\n";
30             return;
31         }
32         cur = cur->next;
33     }
34     std::cout << "The search has failed\n";
35 }
```

```

● // Problem 1:
38  ✓ inline int seqARRcomp(int arr[],int n,int key) {
39      int comparisons= 0;
40  ✓   for (int i= 0; i< n; i++) {
41      |     comparisons++;
42      |     if (arr[i] == key) return comparisons;
43      | }
44      return -1; // not found
45  }
46
47  template<typename T>
48  ✓ int seqLLcomp(Node<T>* head, T key) {
49      int comparisons = 0;
50      Node<T>* cur = head;
51  ✓   while (cur != nullptr) {
52      |     ++comparisons;
53      |     if (cur->data == key) return comparisons;
54      |     cur = cur->next;
55      | }
56      return -1;
57  }
58
59  // Problem 2: count occurrences
60  ✓ inline int countOCarr(int arr[], int n, int key) {
61      int count = 0;
62      for (int i = 0; i < n; i++) if (arr[i] == key) ++count;
63      return count;
64  }
65

```

```

66     template<typename T>
67     ✓ int countOCl1(Node<T>* head, T key) {
68         int count = 0;
69         Node<T> *cur = head;
70     ✓ while (cur != nullptr) {
71         |     if (cur->data == key) count++;
72         |     cur = cur->next;
73     }
74     return count;
75 }
76
77 // Insertion sort for the arrays
78 template<typename T>
79 ✓ void insertionSort(T arr[], int n) {
80     ✓ for (int i= 1; i< n; i++) {
81         |     T key = arr[i];
82         |     int j = i- 1;
83     ✓ while (j >= 0 && arr[j] > key) {
84         |     arr[j + 1] = arr[j];
85         |     j--;
86         |     }
87         |     arr[j + 1] = key;
88     }
89 }
90

```

```

● //binary search
92  ✓ inline int binarySearch(int arr[], int n, int key) {
93      int low = 0;
94      int up = n - 1;
95  ✓  while (low <= up) {
96      int mid = low + (up - low) / 2;
97      std::cout << "Iteration: low=" << low << " mid=" << mid << " high=" << up
98          << " midValue=" << arr[mid] << "\n";
99  ✓      if (arr[mid] == key) {
100          std::cout << "Search element is found at index " << mid << "\n";
101          return mid;
102  ✓      } else if (key < arr[mid]) {
103          up = mid - 1;
104      } else {
105          low = mid + 1;
106      }
107  }
108  std::cout << "Search element is not found\n";
109  return -1;
110  }
111
112  // Recursive binary search
113  ✓ inline int binary_search_recursive_impl(int arr[], int low, int high, int key) {
114      if (low > high) return -1;
115      int mid = low + (high - low) / 2;
116      if (arr[mid] == key) return mid;
117  ☞ if (key < arr[mid]) return binary_search_recursive_impl(arr, low, high: mid - 1, key);
118  ☞ return binary_search_recursive_impl(arr, low: mid + 1, high, key);
119  }
120

```

```

121  ✓ inline int binarySearchRecursive(int arr[], int n, int key) {
      return binary_search_recursive_impl(arr, low: 0, high: n - 1, key);
123  }
124
125  // getMiddle for linked-list binary search
126  template<typename T>
127  ✓ Node<T>* getMiddle(Node<T>* start, Node<T>* last) {
128      if (start == nullptr) return nullptr;
129      Node<T>* slow = start;
130      Node<T>* fast = start;
131  ✓   while (fast != last && fast->next != last) {
132       |   slow = slow->next;
133       |   fast = fast->next->next;
134       |   }
135      return slow;
136  }
137
138  // Binary search on linked list (list must be sorted ascending)
139  template <typename T>
140  ✓ Node<T>* binarySearchLL(Node<T>* start, T findData) {
141      Node<T>* last = nullptr;
142  ✓   while (start != last) {
143       |   Node<T>* mid = getMiddle(start, last);
144       |   if (mid == nullptr) break;
145  ✓   |   if (mid->data == findData) {
146       |   |   std::cout << "The search is success\n";
147       |   |   return mid;
148  ✓   |   } else if (mid->data < findData) {
149       |   |   start = mid->next;
150       |   } else {
151       |   |   last = mid;
152  ✓   |   }
153      }
154      std::cout << "The search has failed\n";
155      return nullptr;
156  }

```

OUTPUT

```
"C:\Users\royga\CLionProjects\untitled1\H0A6p1\SUPPLEMENTARY\Sup main.exe"
Problem 1 list (array):
15 18 2 19 18 0 8 14 19 14
Linked list contents:
15 18 2 19 18 0 8 14 19 14

Problem 1: Sequential search for key 18
Array: comparisons to match = 2
Linked list: comparisons to match = 2

Process finished with exit code 0
|
```

PROBLEM 2:**CODE**

```
88     std::cout << "\n=====";
89     std::cout << "\nPROBLEM 2 - COUNT OCCURRENCES\n";
90     std::cout << "Search element: " << key << "\n";
91
92     // Count occurrences in the array.
93     int arrayCount = countOCCarr(data, n: max_size, key);
94
95     std::cout << "Array occurrences: " << arrayCount << "\n";
96
97     // Count occurrences in the linked list.
98     int listCount = countOCLl(head, key);
99
100    std::cout << "Linked List occurrences: " << listCount << "\n";
101    return 0;
102
103    // cleanup
104    delete_list(head);
105 }
```

OUTPUT

```
C:\Users\ravqa\CLionProjects\untitled1\H0A6p1\SUPPLEMENTARY\SUP2.exe
Rerun 'SUP2.cpp' Ctrl+F5 (y):
15 18 2 19 18 0 8 14 19 14
Linked list contents:
15 18 2 19 18 0 8 14 19 14

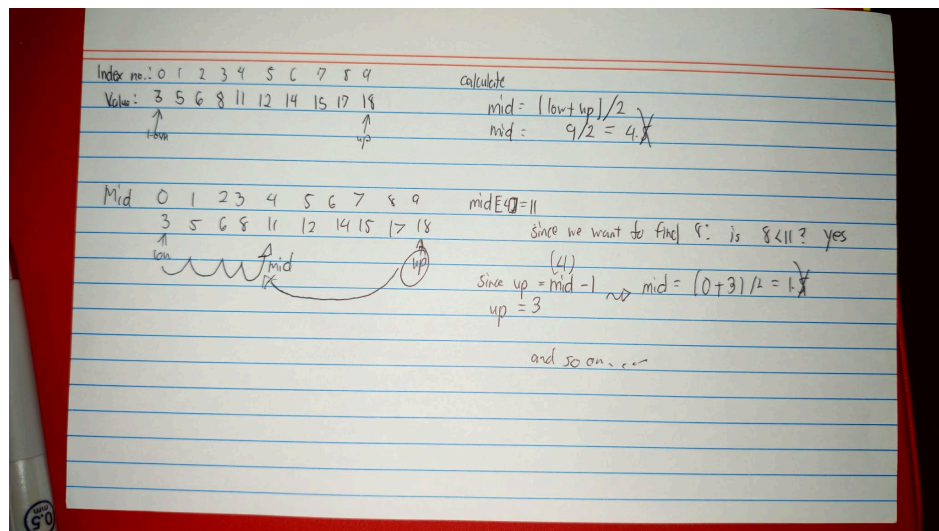
Problem 1: Sequential search for key 18
Array: comparisons to match = 2
Linked list: comparisons to match = 2

=====
PROBLEM 2 - COUNT OCCURRENCES
Search element: 18
Array occurrences: 2
Linked List occurrences: 2

Process finished with exit code 0
```

PROBLEM 3:

DRAWING



Iteration 1

Indx:	0	1	2	3	4	5	6	7	8	9	arr[mid] = 11	if $x < y$ = move right to left
Value:	3	5	6	8	11	12	14	15	17	18	is $8 < 11$? $\times$ up = 3	if $x > y$ = move low to right
				low					up			

Iteration 2:

Indx:	0	1	2	3	4	5	6	7	8	9	arr[mid] = 5	
Value:	3	5	6	8	11	12	14	15	17	18	is $8 > 5$? $\times$ low = 2	
			low		up							

Iteration 3:

Indx:	0	1	2	3	4	5	6	7	8	9	arr[mid] = 6	
Value:	3	5	6	8	11	12	14	15	17	18	is $8 > 6$? $\times$ yes low = 3	
			low	up								

Iteration 4:

Indx:	0	1	2	3	4	5	6	7	8	9	Found 8!
Value:	3	5	6	8	11	12	14	15	17	18	
			low	up							

CODE

```
2 // Created by royga on 8/24/2026.
3 //
4  ✓ #include <iostream>
5     #include "searchingalgo.h"
6
7  ✓ int main()
8     {
9         // Sorted list given in Problem 3.
10        int data[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
11
12        // Search key.
13        int key = 8;
14
15        // Display the list.
16        std::cout << "Sorted List:\n";
17
18        ✓ for (int i = 0; i < 10; i++)
19            {
20                | std::cout << data[i] << " ";
21            }
22
23        std::cout << "\n\nSearching for: "
24        | << key << "\n";
25
26        // Call binary search.
27        binarySearch(data, n: 10, key);
28
29        return 0;
30    }
```

```
158  ✓ int binarySearchIte(int arr[], int n, int key) {
159      // Set the starting boundary.
160      int low = 0;
161
162      // Set the ending boundary.
163      int up = n - 1;
164
165      // Continue while the search range is valid.
166  ✓  while (low <= up)
167      {
168          // Calculate the middle index.
169          int mid = (low + up) / 2;
170
171          // Display the current iteration.
172          std::cout << "\nIteration";
173          std::cout << "\nLow = " << low;
174          std::cout << "\nUp = " << up;
175          std::cout << "\nMid = " << mid;
176          std::cout << "\nMiddle Value = " << arr[mid];
177
178          // Check if the middle value is the key.
179  ✓  if (arr[mid] == key)
180      {
181          // Display successful search.
182          std::cout << "\nSearch element is found!";
183          std::cout << "\nIndex = " << mid << std::endl;
184
185          // Return the index.
186          return mid;
187      }
```

	<pre> 187 } 188 189 // If the key is smaller, search the left side. 190 if (key < arr[mid]) 191 { 192 up = mid - 1; 193 } 194 else 195 { 196 // Otherwise, search the right side. 197 low = mid + 1; 198 } 199 } 200 201 // The key was not found. 202 std::cout << "\nSearch element is not found."; 203 204 return -1; 205 } 206 #endif //UNTITLED1_SEARCHINGALGO_H </pre>
OUTPUT	<pre> C:\Users\royga\CLionProjects\untitled1\H0A6p1\SUPPLEMENTARY\Sup3.exe Sorted List: 3 5 6 8 11 12 14 15 17 18 Searching for: 8 Iteration: low=0 mid=4 high=9 midValue=11 Iteration: low=0 mid=1 high=3 midValue=5 Iteration: low=2 mid=2 high=3 midValue=6 Iteration: low=3 mid=3 high=3 midValue=8 Search element is found at index 3 Process finished with exit code 0 </pre>

PROBLEM 4:

CODE

```
202  ✓ int binarySearchRecursive(int arr[], int low, int up, int key)
203  {
204      // Check if the search range is still valid.
205  ✓   if (low > up){
206      |     // The search element was not found.
207      |     return -1;
208      | }
209
210      // Calculate the middle index.
211      int mid = (low + up) / 2;
212
213      // Display the current recursive step.
214      std::cout << "\nLow = " << low;
215      std::cout << "\nUp = " << up;
216      std::cout << "\nMid = " << mid;
217      std::cout << "\nMiddle Value = " << arr[mid] << "\n";
218
219      // Check if the middle element is the search key.
220  ✓   if (arr[mid] == key){
221      |     // The search was successful.
222      |     std::cout << "Search element is found!";
223      |     std::cout << "\nIndex = " << mid << "\n";
224
225      |     // Return the position of the key.
226      |     return mid;
227      | }
228
229      // If the key is smaller than the middle value,
230      // recursively search the left half.
231  ✓   if (key < arr[mid])
232      | {
233  🔄   |     return binarySearchRecursive(arr, low, up:mid - 1, key);
234      | }
235
236      // If the key is greater than the middle value,
237      // recursively search the right half.
238  🔄   return binarySearchRecursive(arr, low:mid + 1, up, key);
239      }
240      #endif //UNTITLED1_SEARCHINGALGO_H
```

Main

```
1  ▶ //
2    // Created by royga on 8/25/2026.
3    //
4    // Sorted list given in the activity.
5
6  ▾ #include <iostream>
7  #include "searchingalgo.h"
8  ▾ int main() {
9      int data[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
10     // Search element.
11     int key = 8;
12     // Display the sorted list.
13     std::cout << "Sorted List:\n";
14
15     ▾ for (int i = 0; i < 10; i++){
16         |     std::cout << data[i] << " ";
17     }
18
19     std::cout << "\n\nSearching for: "<< key << "\n";
20
21     // Call the recursive binary search.
22     int result = binarySearchRecursive(data, low: 0, up: 9, key);
23
24     // Check whether the search was unsuccessful.
25     ▾ if (result == -1){
26         |     std::cout << "\nSearch element is not found.\n";
27     }
28
29     return 0;
30 }
```

OUTPUT

```
C:\Users\royga\CLionProjects\untitled1\H0A6p1\SUPPLEMENTARY\sup4.exe
Sorted List:
3 5 6 8 11 12 14 15 17 18

Searching for: 8

Low = 0
Up = 9
Mid = 4
Middle Value = 11

Low = 0
Up = 3
Mid = 1
Middle Value = 5

Low = 2
Up = 3
Mid = 2
Middle Value = 6

Low = 3
Up = 3
Mid = 3
Middle Value = 8
Search element is found!
Index = 3

Process finished with exit code 0
```

CONCLUSION:

From all of the procedure from this HOA, I have learned that, searching methods or functions using a linked list or even binary search, requires the loop or the function to iterate or flow like from the start to end then reach the conclusion that it will create. In binary search, we must first arrange the data, whether in an array or linked list, from the lowest value to the highest, and so far, I only understand the integers, but I wonder on how it will work on a characters, even strings.

Overall, the activity has taught me about linear and binary searching is arrays and linked lists, although there is a recursion, it is basically just a loop until fulfillment. It also made me understand how different searching

techniques can be used appropriately on different scenarios or organization of data? Like on linear searching, it can be just any kind of formation, chronological or not, but in binary, it is a requirement for it to be working.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: None

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: None

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: None

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.



Please have the honor of pledge in the comment section or at the bottom of your activity document/exam or any form of file submission.



I affirm that I will not give or receive any unauthorized help on this activity/exam and that all work will be my own.